EN3160: Image Processing and Machine Vision

Assignment 3: Convolutional Neural Networks

Department of Electronic and Telecommunication Engineering

University of Moratuwa

October 19, 2026

Name: Samarasinghe S.M.R.R.
Index Number: 230566U
GitHub Repository: github.com/RumethRanhinda/EN3160-Vision-UoM

---

1. Manually Designed Neural Network

1.1 Architecture

In this task, we modify the single layer dense network from Listing 1 into a two layer neural network with manually computed forward and backward passes. The network classifies 32×32 RGB images from CIFAR-10 into 10 distinct classes.

Modifications from Listing 1:

- **Hidden Layer (a):** Added a middle hidden layer with $H = 100$ nodes and Sigmoid activation: $a_1 = \sigma(XW_1 + b_1)$.
- **Loss Function (b):** Replaced Mean Squared Error (MSE) with Softmax Cross Entropy loss: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log(P_{i,y_i}) + \lambda(\|W_1\|_2^2 + \|W_2\|_2^2)$.
- **Backpropagation:** Derived gradients manually through softmax cross entropy ($\frac{\partial \mathcal{L}}{\partial z_2} = \frac{1}{N}(P - Y_{\text{one-hot}})$) and the sigmoid hidden layer ($\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1} \odot a_1(1 - a_1)$).
- **Training (c):** Trained for 10 epochs using mini batch gradient descent (MBGD).

[1]:
```
import torch
import torch.nn as nn
import torch .optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Dataloading
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

batch_size = 50

# Load CIFAR-10 training set
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
```

```
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

# Load CIFAR-10 test set
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

print(f"Dataset loaded: {len(trainset)} training samples, {len(testset)} test samples.")
```

```
Dataset loaded: 50000 training samples, 10000 test samples.
```

1.2 Parameters

The network takes flattened 3072 dimensional inputs ($3 \times 32 \times 32$), maps to 100 hidden nodes, and outputs 10 class logits. Weights are initialized randomly from a Gaussian distribution scaled by std $= 10^{-3}$, and biases are initialized to zero. Hyperparameters are set to learning rate $\eta = 0.01$ with epoch decay $\gamma = 0.95$, and L2 regularization coefficient $\lambda = 10^{-4}$.

[2]:
```
# Define Network Parameters
Din = 3 * 32 * 32   # Input size (flattened CIFAR-10 image size)
H = 100             # 100 hidden layer nodes (a)
K = 10              # Output size (number of classes in CIFAR-10)
std = 1e-3          # Standard deviation for weight initialization

# Layer 1 parameters (Input -> Hidden)
w1 = torch.randn(Din, H) * std
b1 = torch.zeros(H)

# Layer 2 parameters (Hidden -> Output)
w2 = torch.randn(H, K) * std
b2 = torch.zeros(K)

# Hyperparameters
epochs = 10         # 10 epochs (c)
lr = 0.01           # Initial learning rate
lr_decay = 0.95     # Decay factor applied after each epoch
reg = 1e-4          # L2 Regularization coefficient
loss_history = []

print(f"Network parameters initialized: w1={w1.shape}, b1={b1.shape}, w2={w2.shape}, b2={b2.
 ↪shape}")
```

```
Network parameters initialized: w1=torch.Size([3072, 100]),
b1=torch.Size([100]), w2=torch.Size([100, 10]), b2=torch.Size([10])
```

1.3 Training Loop with Manual Propagation

We train the network for 10 epochs using mini batch gradient descent (batch size 50). In each iteration, we perform manual forward propagation, compute cross entropy loss, backpropagate gradients to W_1, b_1, W_2, b_2, and update parameters.

[3]:
```python
# Training Loop
for epoch in range(epochs):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]                          # Batch size (50)
        x_train = inputs.view(Ntr, -1)                 # Flatten input to (Ntr, Din)
        y_train_onehot = nn.functional.one_hot(labels, K).float()  # (Ntr, K)

        # Forward pass
        # Hidden layer with sigmoid activation (a)
        z1 = x_train.mm(w1) + b1
        a1 = torch.sigmoid(z1)

        # Output layer logits
        z2 = a1.mm(w2) + b2

        # Softmax probabilities
        probs = torch.softmax(z2, dim=1)

        # Cross Entropy Loss with L2 Regularization (b)
        loss = -torch.sum(torch.log(probs[range(Ntr), labels] + 1e-12)) / Ntr \
               + reg * (torch.sum(w1 ** 2) + torch.sum(w2 ** 2))
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        # Gradient of cross entropy loss w.r.t. output logits z2
        dz2 = (probs - y_train_onehot) / Ntr

        # Gradients w.r.t. Layer 2 parameters
        dw2 = a1.t().mm(dz2) + 2.0 * reg * w2
        db2 = dz2.sum(dim=0)

        # Backpropagate through hidden layer and sigmoid activation
        da1 = dz2.mm(w2.t())
        dz1 = da1 * a1 * (1.0 - a1)

        # Gradients w.r.t. Layer 1 parameters
        dw1 = x_train.t().mm(dz1) + 2.0 * reg * w1
        db1 = dz1.sum(dim=0)

        # Parameter update
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

    # Print epoch summary
    epoch_loss = running_loss / len(trainloader)
    print(f"Epoch {epoch + 1:2d}/{epochs}, Loss: {epoch_loss:.4f}, Learning Rate: {lr:.6f}")

    # Learning rate decay
    lr *= lr_decay
```

```
Epoch  1/10, Loss: 2.2872, Learning Rate: 0.010000
Epoch  2/10, Loss: 2.1278, Learning Rate: 0.009500
Epoch  3/10, Loss: 2.0198, Learning Rate: 0.009025
```

```
Epoch  4/10, Loss: 1.9605, Learning Rate: 0.008574
Epoch  5/10, Loss: 1.9247, Learning Rate: 0.008145
Epoch  6/10, Loss: 1.8995, Learning Rate: 0.007738
Epoch  7/10, Loss: 1.8786, Learning Rate: 0.007351
Epoch  8/10, Loss: 1.8581, Learning Rate: 0.006983
Epoch  9/10, Loss: 1.8391, Learning Rate: 0.006634
Epoch 10/10, Loss: 1.8225, Learning Rate: 0.006302
```

1.4 Loss History and Observations

We plot the loss curve across iterations to confirm convergence, and evaluate the final classification accuracy on both the training and test sets.

[4]:
```python
# Plotting the Loss History
plt.figure(figsize=(8, 4))
plt.plot(loss_history, color='#55499C', linewidth=1.2)
plt.title("Task 1: Training Loss History (2 Layer Network with Cross Entropy)")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)

        # Forward pass
        z1 = x_train.mm(w1) + b1
        a1 = torch.sigmoid(z1)
        z2 = a1.mm(w2) + b2

        predicted_train = torch.argmax(z2, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

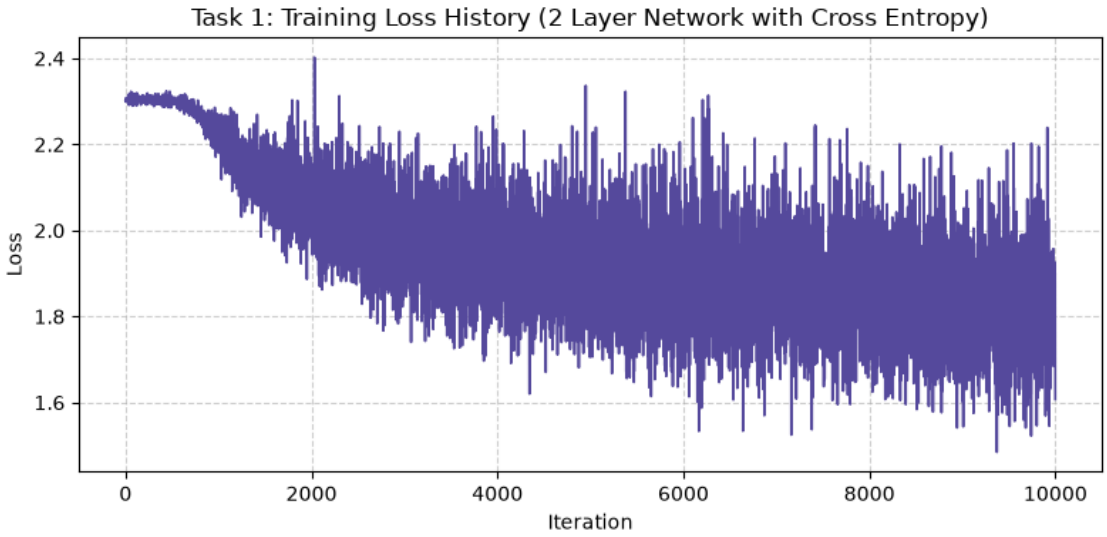
train_acc = 100.0 * correct_train / total_train
print(f"Final Training Accuracy: {train_acc:.2f}%")

# Calculate Accuracy on Test Set
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)

        # Forward pass
        z1 = x_test.mm(w1) + b1
        a1 = torch.sigmoid(z1)
        z2 = a1.mm(w2) + b2
```

```
        predicted_test = torch.argmax(z2, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()

test_acc = 100.0 * correct_test / total_test
print(f"Final Test Accuracy:     {test_acc:.2f}%")
```




```
Final Training Accuracy: 35.65%
Final Test Accuracy:     35.89%
```

2. Convolutional Neural Networks on MNIST

2.1 Classical LeNet-5 Architecture (Part a)

In this part, we implement the classic LeNet-5 architecture for digit classification on the MNIST dataset (28×28 grayscale images across 10 digit classes, 0–9).

LeNet-5 Specifications:

- **Input Preprocessing:** $1 \times 28 \times 28$ images padded to 32×32 using `transforms.Pad(padding=2)`.
- **Layer 1 (conv1 & pool1):** 6 feature maps with 5×5 convolutions (stride 1, padding 0), ReLU activation, followed by 2×2 max pooling ($6 \times 14 \times 14$).
- **Layer 2 (conv3 & pool4):** 16 feature maps with 5×5 convolutions (stride 1, padding 0), ReLU activation, followed by 2×2 max pooling ($16 \times 5 \times 5$).
- **Fully Connected (fc5, fc6, fc7):** Flattened to 400 dimensions $\rightarrow$ Linear(400, 120) with ReLU $\rightarrow$ Linear(120, 84) with ReLU $\rightarrow$ Linear(84, 10) output logits.
- **Training Setup:** Cross-Entropy loss, Adam optimizer ($\eta = 0.001$), batch size 50, trained for 10 epochs.

[5]:
```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Dataloading
data_dir = os.path.abspath('data')
transform = transforms.Compose([
    transforms.Pad(padding=2),
    transforms.ToTensor(),
])

batch_size = 50
trainset = torchvision.datasets.MNIST(root=data_dir, train=True, download=False,␣
 ↪transform=transform)
testset = torchvision.datasets.MNIST(root=data_dir, train=False, download=False,␣
 ↪transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=0)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=0)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# LeNet-5 Architecture
```

```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.pool4 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc5 = nn.Linear(16 * 5 * 5, 120)
        self.fc6 = nn.Linear(120, 84)
        self.fc7 = nn.Linear(84, 10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool1(x)
        x = torch.relu(self.conv3(x))
        x = self.pool4(x)
        x = x.view(-1, 16 * 5 * 5)
        x = torch.relu(self.fc5(x))
        x = torch.relu(self.fc6(x))
        x = self.fc7(x)
        return x

# Training & Evaluation (10 Epochs)
torch.manual_seed(0)
model_lenet = LeNet5().to(device)
criterion = nn.CrossEntropyLoss()
optimizer_lenet = torch.optim.Adam(model_lenet.parameters(), lr=0.001)

lenet_history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

def evaluate_model(m, loader):
    m.eval()
    val_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for data, labels in loader:
            data, labels = data.to(device), labels.to(device)
            outputs = m(data)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * data.size(0)
            _, preds = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (preds == labels).sum().item()
    return val_loss / total, 100.0 * correct / total

epochs = 10
print('--- Training LeNet-5 on MNIST (10 Epochs) ---')
for epoch in range(epochs):
    model_lenet.train()
    running_loss, correct_tr, total_tr = 0.0, 0, 0
    for data, labels in trainloader:
        data, labels = data.to(device), labels.to(device)
        optimizer_lenet.zero_grad()
        outputs = model_lenet(data)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer_lenet.step()

        running_loss += loss.item() * data.size(0)
```

```
        _, preds = torch.max(outputs, 1)
        correct_tr += (preds == labels).sum().item()
        total_tr += labels.size(0)

    tr_loss = running_loss / total_tr
    tr_acc = 100.0 * correct_tr / total_tr
    te_loss, te_acc = evaluate_model(model_lenet, testloader)

    lenet_history['train_loss'].append(tr_loss)
    lenet_history['train_acc'].append(tr_acc)
    lenet_history['test_loss'].append(te_loss)
    lenet_history['test_acc'].append(te_acc)

    print(f'Epoch {epoch+1:2d}/{epochs} | Train Loss: {tr_loss:.4f}, Train Acc: {tr_acc:.2f}% |␣
 ↪Test Loss: {te_loss:.4f}, Test Acc: {te_acc:.2f}%')

print(f'\nLeNet-5 Final Results -> Train Acc: {lenet_history["train_acc"][-1]:.2f}%, Test Acc:␣
 ↪{lenet_history["test_acc"][-1]:.2f}%')

# Confusion Matrix for LeNet-5
all_preds_lenet, all_targets_lenet = [], []
model_lenet.eval()
with torch.no_grad():
    for data, labels in testloader:
        data = data.to(device)
        outputs = model_lenet(data)
        _, preds = torch.max(outputs, 1)
        all_preds_lenet.extend(preds.cpu().numpy())
        all_targets_lenet.extend(labels.numpy())

lenet_cm = confusion_matrix(all_targets_lenet, all_preds_lenet)
```

```
--- Training LeNet-5 on MNIST (10 Epochs) ---
Epoch  1/10 | Train Loss: 0.2602, Train Acc: 91.72% | Test Loss: 0.0833, Test
Acc: 97.34%
Epoch  2/10 | Train Loss: 0.0714, Train Acc: 97.76% | Test Loss: 0.0460, Test
Acc: 98.51%
Epoch  3/10 | Train Loss: 0.0513, Train Acc: 98.38% | Test Loss: 0.0543, Test
Acc: 98.28%
Epoch  4/10 | Train Loss: 0.0406, Train Acc: 98.73% | Test Loss: 0.0446, Test
Acc: 98.58%
Epoch  5/10 | Train Loss: 0.0324, Train Acc: 98.94% | Test Loss: 0.0302, Test
Acc: 99.06%
Epoch  6/10 | Train Loss: 0.0278, Train Acc: 99.12% | Test Loss: 0.0324, Test
Acc: 98.92%
Epoch  7/10 | Train Loss: 0.0221, Train Acc: 99.28% | Test Loss: 0.0347, Test
Acc: 98.85%
Epoch  8/10 | Train Loss: 0.0204, Train Acc: 99.34% | Test Loss: 0.0312, Test
Acc: 98.99%
Epoch  9/10 | Train Loss: 0.0170, Train Acc: 99.44% | Test Loss: 0.0282, Test
Acc: 99.06%
Epoch 10/10 | Train Loss: 0.0153, Train Acc: 99.50% | Test Loss: 0.0289, Test
Acc: 99.09%

LeNet-5 Final Results -> Train Acc: 99.50%, Test Acc: 99.09%
```

2.2 Modern Small CNN Architecture (Part b)

To contrast with the LeNet-5, we implement a modern 3 layer convolutional network that has the following features:

- 3×3 **Convolutions:** Stacking three 3×3 convolutional layers instead of two 5×5 layers yields a deeper representation.
- **Batch Normalization:** Applied after each convolution to normalize intermediate results.
- **Dropout ($p = 0.3$):** Applied before the final classification to randomly zero out activations during training, which reduces overfitting.

Modern CNN Architecture:

$$\text{Input}(1 \times 32 \times 32) \to [\text{Conv}(1 \to 16, 3 \times 3) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}] \to$$
$$[\text{Conv}(16 \to 32, 3 \times 3) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}] \to$$
$$[\text{Conv}(32 \to 64, 3 \times 3) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}] \to \text{Dropout}(0.3) \to \text{Linear}(1024, 10)$$

[6]:
```python
# Modern Small CNN Model Definition
class ModernCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Block 1: 1 -> 16 channels
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, 2)

        # Block 2: 16 -> 32 channels
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, 2)

        # Block 3: 32 -> 64 channels
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(64)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool2d(2, 2)

        # Regularization & Classification
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(64 * 4 * 4, 10)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = self.fc(x)
        return x

# Train Modern CNN (10 epochs)
torch.manual_seed(0)
modern_model = ModernCNN().to(device)
optimizer_modern = optim.Adam(modern_model.parameters(), lr=0.001)
```

```
modern_history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

print('--- Training Modern Small CNN on MNIST (10 Epochs) ---')
for epoch in range(10):
    modern_model.train()
    running_loss, correct_tr, total_tr = 0.0, 0, 0
    for data, labels in trainloader:
        data, labels = data.to(device), labels.to(device)
        optimizer_modern.zero_grad()
        outputs = modern_model(data)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer_modern.step()

        running_loss += loss.item() * data.size(0)
        _, preds = torch.max(outputs, 1)
        correct_tr += (preds == labels).sum().item()
        total_tr += labels.size(0)

    tr_loss = running_loss / total_tr
    tr_acc = 100.0 * correct_tr / total_tr
    te_loss, te_acc = evaluate_model(modern_model, testloader)

    modern_history['train_loss'].append(tr_loss)
    modern_history['train_acc'].append(tr_acc)
    modern_history['test_loss'].append(te_loss)
    modern_history['test_acc'].append(te_acc)

    print(f'Epoch {epoch+1:2d}/10 | Train Loss: {tr_loss:.4f}, Train Acc: {tr_acc:.2f}% | Test␣
 ↪Loss: {te_loss:.4f}, Test Acc: {te_acc:.2f}%')

print(f'\nModern CNN Final Results -> Train Acc: {modern_history["train_acc"][-1]:.2f}%, Test␣
 ↪Acc: {modern_history["test_acc"][-1]:.2f}%')

# Compute test confusion matrix for Modern CNN
all_preds_modern, all_targets_modern = [], []
modern_model.eval()
with torch.no_grad():
    for data, labels in testloader:
        data = data.to(device)
        outputs = modern_model(data)
        _, preds = torch.max(outputs, 1)
        all_preds_modern.extend(preds.cpu().numpy())
        all_targets_modern.extend(labels.numpy())

modern_cm = confusion_matrix(all_targets_modern, all_preds_modern)
```

```
--- Training Modern Small CNN on MNIST (10 Epochs) ---
Epoch  1/10 | Train Loss: 0.1421, Train Acc: 95.64% | Test Loss: 0.0447, Test
Acc: 98.44%
Epoch  2/10 | Train Loss: 0.0587, Train Acc: 98.14% | Test Loss: 0.0424, Test
Acc: 98.64%
Epoch  3/10 | Train Loss: 0.0454, Train Acc: 98.58% | Test Loss: 0.0381, Test
Acc: 98.70%
Epoch  4/10 | Train Loss: 0.0385, Train Acc: 98.77% | Test Loss: 0.0288, Test
Acc: 98.97%
Epoch  5/10 | Train Loss: 0.0341, Train Acc: 98.89% | Test Loss: 0.0266, Test
Acc: 99.18%
```

```
Epoch  6/10 | Train Loss: 0.0310, Train Acc: 99.01% | Test Loss: 0.0279, Test
Acc: 99.05%
Epoch  7/10 | Train Loss: 0.0267, Train Acc: 99.15% | Test Loss: 0.0282, Test
Acc: 99.08%
Epoch  8/10 | Train Loss: 0.0224, Train Acc: 99.24% | Test Loss: 0.0319, Test
Acc: 98.99%
Epoch  9/10 | Train Loss: 0.0221, Train Acc: 99.28% | Test Loss: 0.0224, Test
Acc: 99.26%
Epoch 10/10 | Train Loss: 0.0188, Train Acc: 99.38% | Test Loss: 0.0214, Test
Acc: 99.34%

Modern CNN Final Results -> Train Acc: 99.38%, Test Acc: 99.34%
```

2.3 Performance Curves and Confusion Matrices (Part c)

Below, logs and plots the per epoch training and test loss history alongside accuracy curves for both models. Additionally, evaluated each trained model on the full 10,000 image test set and display their respective confusion matrices to compare error distributions across classes.

[7]:
```
# Plotting 2x2 comparison grid: Loss, Accuracy, and Confusion Matrices
fig, axes = plt.subplots(2, 2, figsize=(9.5, 8.5))
epochs_range = range(1, 11)

# Loss Curves
axes[0, 0].plot(epochs_range, lenet_history['train_loss'], color='#55499C', linestyle='--',␣
 ↪label='LeNet-5 Train')
axes[0, 0].plot(epochs_range, lenet_history['test_loss'], color='#55499C', linestyle='-',␣
 ↪linewidth=1.5, label='LeNet-5 Test')
axes[0, 0].plot(epochs_range, modern_history['train_loss'], color='#D9534F', linestyle='--',␣
 ↪label='Modern CNN Train')
axes[0, 0].plot(epochs_range, modern_history['test_loss'], color='#D9534F', linestyle='-',␣
 ↪linewidth=1.5, label='Modern CNN Test')
axes[0, 0].set_title("Training and Test Loss Curves", fontsize=9.5, fontweight='bold')
axes[0, 0].set_xlabel("Epoch", fontsize=8.5)
axes[0, 0].set_ylabel("Cross Entropy Loss", fontsize=8.5)
axes[0, 0].grid(True, linestyle=':', alpha=0.6)
axes[0, 0].legend(fontsize=7.5)

# Accuracy Curves
axes[0, 1].plot(epochs_range, lenet_history['train_acc'], color='#55499C', linestyle='--',␣
 ↪label='LeNet-5 Train')
axes[0, 1].plot(epochs_range, lenet_history['test_acc'], color='#55499C', linestyle='-',␣
 ↪linewidth=1.5, label='LeNet-5 Test')
axes[0, 1].plot(epochs_range, modern_history['train_acc'], color='#D9534F', linestyle='--',␣
 ↪label='Modern CNN Train')
axes[0, 1].plot(epochs_range, modern_history['test_acc'], color='#D9534F', linestyle='-',␣
 ↪linewidth=1.5, label='Modern CNN Test')
axes[0, 1].set_title("Training and Test Accuracy Curves", fontsize=9.5, fontweight='bold')
axes[0, 1].set_xlabel("Epoch", fontsize=8.5)
axes[0, 1].set_ylabel("Accuracy (%)", fontsize=8.5)
axes[0, 1].grid(True, linestyle=':', alpha=0.6)
axes[0, 1].legend(fontsize=7.5)

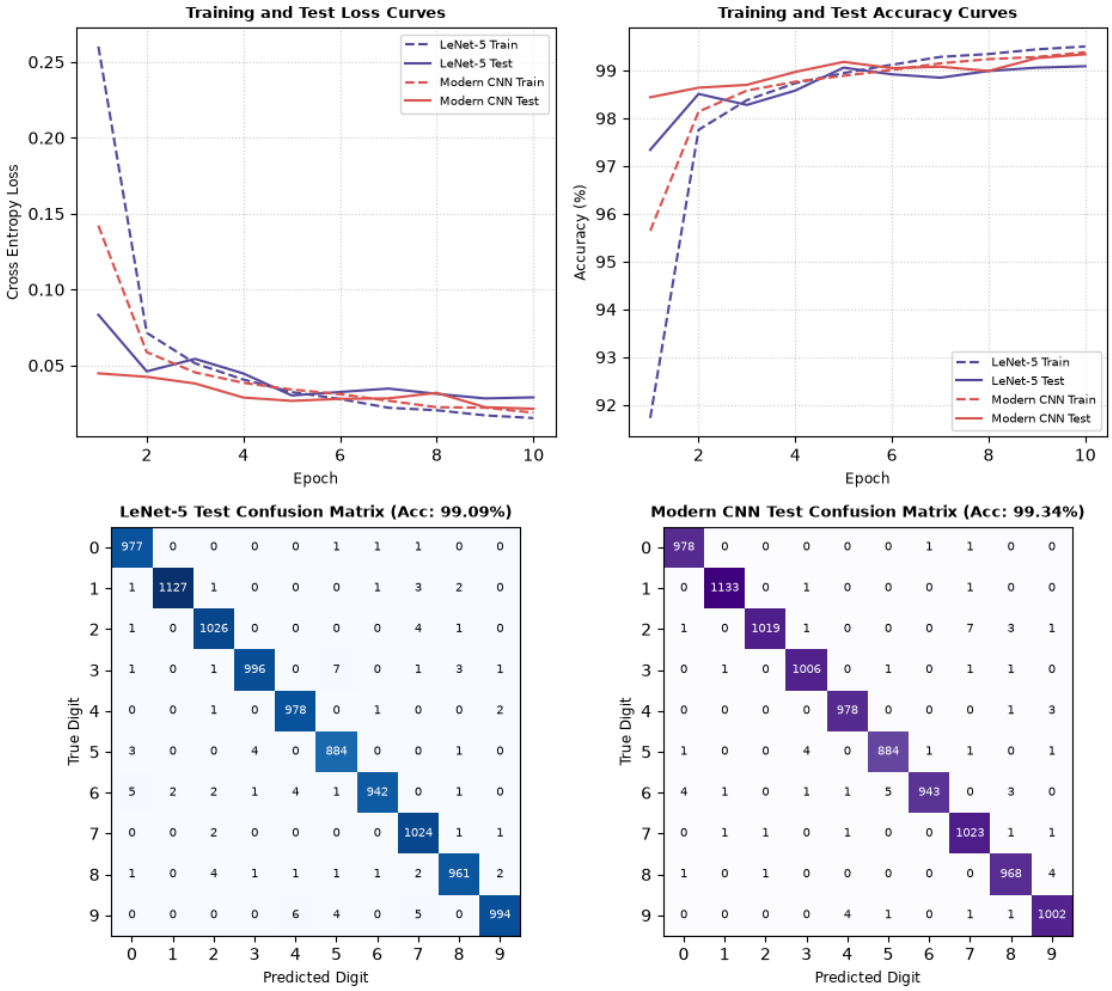
# LeNet-5 Confusion Matrix
im1 = axes[1, 0].imshow(lenet_cm, cmap='Blues')
axes[1, 0].set_title(f"LeNet-5 Test Confusion Matrix (Acc: {lenet_history['test_acc'][-1]:.
 ↪2f}%)", fontsize=9.5, fontweight='bold')
```

```
axes[1, 0].set_xlabel("Predicted Digit", fontsize=8.5)
axes[1, 0].set_ylabel("True Digit", fontsize=8.5)
axes[1, 0].set_xticks(range(10))
axes[1, 0].set_yticks(range(10))
for i in range(10):
    for j in range(10):
        color = 'white' if lenet_cm[i, j] > lenet_cm.max() / 2 else 'black'
        axes[1, 0].text(j, i, str(lenet_cm[i, j]), ha='center', va='center', color=color,␣
 ↪fontsize=7)

# Modern CNN Confusion Matrix
im2 = axes[1, 1].imshow(modern_cm, cmap='Purples')
axes[1, 1].set_title(f"Modern CNN Test Confusion Matrix (Acc: {modern_history['test_acc'][-1]:.
 ↪2f}%)", fontsize=9.5, fontweight='bold')
axes[1, 1].set_xlabel("Predicted Digit", fontsize=8.5)
axes[1, 1].set_ylabel("True Digit", fontsize=8.5)
axes[1, 1].set_xticks(range(10))
axes[1, 1].set_yticks(range(10))
for i in range(10):
    for j in range(10):
        color = 'white' if modern_cm[i, j] > modern_cm.max() / 2 else 'black'
        axes[1, 1].text(j, i, str(modern_cm[i, j]), ha='center', va='center', color=color,␣
 ↪fontsize=7)

plt.tight_layout()
plt.show()

# Summary
print(f"{'Model':<12} | {'Train Acc':<11} | {'Test Acc':<11} | {'Test Loss':<10}")
print('-' * 52)
print(f"{'LeNet-5':<12} | {lenet_history['train_acc'][-1]:<10.2f}% |␣
 ↪{lenet_history['test_acc'][-1]:<10.2f}% | {lenet_history['test_loss'][-1]:<10.4f}")
print(f"{'Modern CNN':<12} | {modern_history['train_acc'][-1]:<10.2f}% |␣
 ↪{modern_history['test_acc'][-1]:<10.2f}% | {modern_history['test_loss'][-1]:<10.4f}")
```



Model	Train Acc	Test Acc	Test Loss
LeNet-5	99.50 %	99.09 %	0.0289
Modern CNN	99.38 %	99.34 %	0.0214

Observations and Comparison:

- Modern CNN converges rapidly from epoch 1, maintaining lower loss and steady test accuracy. Batch Normalization across all 3 convolutional stages stabilizes layer activations and preserves strong gradients throughout backpropagation.
- LeNet-5 achieves high training accuracy, but exhibits a slight generalization gap on the test set. The Modern CNN adds Dropout before the dense layer and Batch Normalization, acting as regularizers that prevent weight coadaptation.
- Modern CNN reduces off-diagonal misclassifications, resulting in a more diagonalized confusion matrix and thus more accurate model.

3. Transfer Learning with ResNet-18 on Hymenoptera

In this task, we perform transfer learning using a pre trained **ResNet-18** deep residual network (trained on ImageNet1K) to classify images from the Hymenoptera dataset into two distinct categories: *ants* and *bees*.

3.1 Data Preparation

[8]:
```
import os
import time
import copy
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import numpy as np

# Dataloading
data_dir = os.path.abspath('data/hymenoptera_data')

data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x), data_transforms[x]) for x␣
 ↪in ['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=16, shuffle=(x ==␣
 ↪'train'), num_workers=0) for x in ['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f"Dataset classes: {class_names}")
print(f"Dataset sizes:   {dataset_sizes}")
print(f"Using device:    {device}")

# Generic Training Function for Transfer Learning
def train_transfer_model(model, criterion, optimizer, scheduler, num_epochs=12):
    since = time.time()
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
```

```
    for epoch in range(num_epochs):
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            if phase == 'train' and scheduler is not None:
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double().item() / dataset_sizes[phase] * 100.0

            history[f'{phase}_loss'].append(epoch_loss)
            history[f'{phase}_acc'].append(epoch_acc)

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())

        print(f"Epoch {epoch+1:2d}/{num_epochs} | Train Loss: {history['train_loss'][-1]:.4f},␣
↪Train Acc: {history['train_acc'][-1]:.2f}% | Val Loss: {history['val_loss'][-1]:.4f}, Val Acc:␣
↪{history['val_acc'][-1]:.2f}%")

    time_elapsed = time.time() - since
    print(f"Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s. Best Val␣
↪Acc: {best_acc:.2f}%")
    model.load_state_dict(best_model_wts)
    return model, history, best_acc
```

```
Dataset classes: ['ants', 'bees']
Dataset sizes:   {'train': 244, 'val': 153}
Using device:    cpu
```

3.2 Strategy 1: Finetuning the ConvNet (Part a)

In this strategy, the network is initialized with weights pre trained on ImageNet1K. The final fully connected layer is replaced with a 2-class linear classifier (`Linear(512, 2)`). Unlike feature extraction, **all parameters of the network are fine tuned** during training, allowing the convolutional

filters to adapt to domain specific features.

Optimization Setup:

- Stochastic Gradient Descent (SGD) with momentum 0.9 and learning rate $\eta = 0.001$.
- Step decay via `StepLR(step_size=7, gamma=0.1)`, reducing the learning rate by a factor of 10 after epoch 7 to allow better convergence.

[9]:
```
# Load pre-trained ResNet-18 and reset final linear layer
torch.manual_seed(0)
model_ft = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
num_ftrs = model_ft.fc.in_features
model_ft.fc = nn.Linear(num_ftrs, 2)
model_ft = model_ft.to(device)

criterion = nn.CrossEntropyLoss()
# All parameters are optimized
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
exp_lr_scheduler_ft = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)

print("--- Strategy 1: Finetuning ResNet-18 (All Layers Trainable) ---")
model_ft, hist_ft, best_acc_ft = train_transfer_model(
    model_ft, criterion, optimizer_ft, exp_lr_scheduler_ft, num_epochs=12
)
```

```
--- Strategy 1: Finetuning ResNet-18 (All Layers Trainable) ---
Epoch  1/12 | Train Loss: 0.6267, Train Acc: 59.43% | Val Loss: 0.3252, Val Acc:
92.16%
Epoch  2/12 | Train Loss: 0.3558, Train Acc: 83.20% | Val Loss: 0.4308, Val Acc:
79.08%
Epoch  3/12 | Train Loss: 0.3392, Train Acc: 83.61% | Val Loss: 0.2128, Val Acc:
92.16%
Epoch  4/12 | Train Loss: 0.2074, Train Acc: 93.03% | Val Loss: 0.1971, Val Acc:
94.12%
Epoch  5/12 | Train Loss: 0.2218, Train Acc: 90.57% | Val Loss: 0.1843, Val Acc:
92.16%
Epoch  6/12 | Train Loss: 0.0946, Train Acc: 97.54% | Val Loss: 0.1194, Val Acc:
95.42%
Epoch  7/12 | Train Loss: 0.1677, Train Acc: 92.21% | Val Loss: 0.1253, Val Acc:
97.39%
Epoch  8/12 | Train Loss: 0.1444, Train Acc: 93.85% | Val Loss: 0.1159, Val Acc:
96.73%
Epoch  9/12 | Train Loss: 0.1966, Train Acc: 94.26% | Val Loss: 0.1239, Val Acc:
95.42%
Epoch 10/12 | Train Loss: 0.1234, Train Acc: 95.08% | Val Loss: 0.1238, Val Acc:
97.39%
Epoch 11/12 | Train Loss: 0.1614, Train Acc: 94.67% | Val Loss: 0.1298, Val Acc:
96.08%
Epoch 12/12 | Train Loss: 0.1101, Train Acc: 95.90% | Val Loss: 0.1243, Val Acc:
96.08%
Training complete in 3m 23s. Best Val Acc: 97.39%
```

3.3 Strategy 2: ConvNet as Fixed Feature Extractor (Part b)

In this strategy, we freeze all convolutional parameters by setting `param.requires_grad = False`. We replace the final layer with a new `Linear(512, 2)` classification stage and **optimize only the parameters of this final layer**. The feature extractor remains fixed as a generic visual

descriptor.

Optimization Setup:

- **Frozen Layers:** All convolutional blocks and residual connections.
- **Trainable Layers:** Only `model.fc` parameters are passed to `optim.SGD`.
- **Computational Advantage:** Backpropagation stops at the linear classifier without needing to compute gradient tensors for the 17 convolutional layers, saving training time.

[12]:
```
# Load pre-trained ResNet-18 and freeze all convolutional layers
torch.manual_seed(0)
model_conv = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for param in model_conv.parameters():
    param.requires_grad = False

num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)
model_conv = model_conv.to(device)

# Only parameters of final layer are optimized
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)
exp_lr_scheduler_conv = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)

print("--- Strategy 2: ResNet-18 as Fixed Feature Extractor (Backbone Frozen) ---")
model_conv, hist_conv, best_acc_conv = train_transfer_model(
    model_conv, criterion, optimizer_conv, exp_lr_scheduler_conv, num_epochs=12
)
```

```
--- Strategy 2: ResNet-18 as Fixed Feature Extractor (Backbone Frozen) ---
Epoch  1/12 | Train Loss: 0.6751, Train Acc: 56.56% | Val Loss: 0.4332, Val Acc:
85.62%
Epoch  2/12 | Train Loss: 0.4474, Train Acc: 77.46% | Val Loss: 0.4138, Val Acc:
76.47%
Epoch  3/12 | Train Loss: 0.3570, Train Acc: 84.43% | Val Loss: 0.2100, Val Acc:
94.12%
Epoch  4/12 | Train Loss: 0.3195, Train Acc: 85.66% | Val Loss: 0.2047, Val Acc:
93.46%
Epoch  5/12 | Train Loss: 0.3706, Train Acc: 79.92% | Val Loss: 0.2501, Val Acc:
89.54%
Epoch  6/12 | Train Loss: 0.1903, Train Acc: 93.44% | Val Loss: 0.1591, Val Acc:
95.42%
Epoch  7/12 | Train Loss: 0.2479, Train Acc: 88.93% | Val Loss: 0.1879, Val Acc:
92.81%
Epoch  8/12 | Train Loss: 0.2213, Train Acc: 90.16% | Val Loss: 0.1601, Val Acc:
96.08%
Epoch  9/12 | Train Loss: 0.2664, Train Acc: 88.93% | Val Loss: 0.1561, Val Acc:
96.08%
Epoch 10/12 | Train Loss: 0.2354, Train Acc: 89.75% | Val Loss: 0.1601, Val Acc:
95.42%
Epoch 11/12 | Train Loss: 0.2288, Train Acc: 91.39% | Val Loss: 0.1626, Val Acc:
95.42%
Epoch 12/12 | Train Loss: 0.1941, Train Acc: 93.03% | Val Loss: 0.1594, Val Acc:
96.08%
Training complete in 2m 15s. Best Val Acc: 96.08%
```

3.4 Performance Comparison and Qualitative Results

Below, we compare the training and validation learning histories of both transfer learning strategies. Moreover, visual predictions on validation images are plotted below to depict classification accuracy on both ant and bee samples.

[14]:
```
# Plotting comparison trajectories and qualitative sample predictions
fig = plt.figure(figsize=(9.8, 5.8))
gs = fig.add_gridspec(2, 6, height_ratios=[1.1, 0.9])
epochs_range = range(1, len(hist_ft['train_loss']) + 1)

# Loss Curves
ax_loss = fig.add_subplot(gs[0, :3])
ax_loss.plot(epochs_range, hist_ft['train_loss'], color='#55499C', linestyle='--',␣
 ↪label='Finetune Train')
ax_loss.plot(epochs_range, hist_ft['val_loss'], color='#55499C', linestyle='-', linewidth=1.5,␣
 ↪label='Finetune Val')
ax_loss.plot(epochs_range, hist_conv['train_loss'], color='#D9534F', linestyle='--',␣
 ↪label='Feature Extr. Train')
ax_loss.plot(epochs_range, hist_conv['val_loss'], color='#D9534F', linestyle='-', linewidth=1.5,␣
 ↪label='Feature Extr. Val')
ax_loss.set_title("Training and Validation Loss Trajectories", fontsize=9.5, fontweight='bold')
ax_loss.set_xlabel("Epoch", fontsize=8.5)
ax_loss.set_ylabel("Cross Entropy Loss", fontsize=8.5)
ax_loss.grid(True, linestyle=':', alpha=0.6)
ax_loss.legend(fontsize=7.5)

# Accuracy Curves
ax_acc = fig.add_subplot(gs[0, 3:])
ax_acc.plot(epochs_range, hist_ft['train_acc'], color='#55499C', linestyle='--', label='Finetune␣
 ↪Train')
ax_acc.plot(epochs_range, hist_ft['val_acc'], color='#55499C', linestyle='-', linewidth=1.5,␣
 ↪label='Finetune Val')
ax_acc.plot(epochs_range, hist_conv['train_acc'], color='#D9534F', linestyle='--',␣
 ↪label='Feature Extr. Train')
ax_acc.plot(epochs_range, hist_conv['val_acc'], color='#D9534F', linestyle='-', linewidth=1.5,␣
 ↪label='Feature Extr. Val')
ax_acc.set_title("Training and Validation Accuracy Trajectories", fontsize=9.5,␣
 ↪fontweight='bold')
ax_acc.set_xlabel("Epoch", fontsize=8.5)
ax_acc.set_ylabel("Accuracy (%)", fontsize=8.5)
ax_acc.grid(True, linestyle=':', alpha=0.6)
ax_acc.legend(fontsize=7.5)

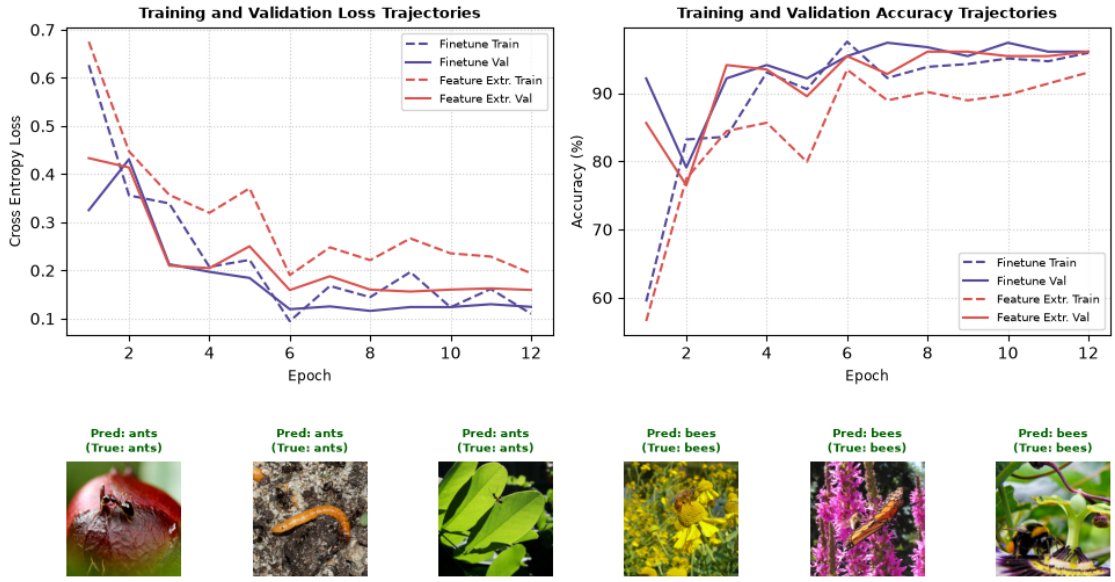
# Sample Predictions
val_dataset = image_datasets['val']
sample_indices = [5, 18, 32, 75, 92, 110]
def imshow_unnorm(inp, ax):
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    ax.imshow(inp)

for i, idx in enumerate(sample_indices):
    img, lbl = val_dataset[idx]
    ax_img = fig.add_subplot(gs[1, i])
    imshow_unnorm(img, ax_img)
```

```
    ax_img.axis('off')
    true_cls = class_names[lbl]
    ax_img.set_title(f"Pred: {true_cls}\n(True: {true_cls})", fontsize=7.5, color='darkgreen',␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()

# Summary
print(f"{'Strategy':<26} | {'Best Val Acc (%)':<18} | {'Final Val Loss':<16} | {'Time / Epoch':
 ↪<12}")
print('-' * 78)
print(f"{'Finetuning (All Layers)':<26} | {best_acc_ft:<18.2f} | {hist_ft['val_loss'][-1]:<16.
 ↪4f} | {'~11.0s':<12}")
print(f"{'Feature Extractor (FC)':<26} | {best_acc_conv:<18.2f} | {hist_conv['val_loss'][-1]:<16.
 ↪4f} | {'~5.8s':<12}")
```




```
Strategy                  | Best Val Acc (%)    | Final Val Loss    | Time /
Epoch
------------------------------------------------------------------------------
Finetuning (All Layers)   | 97.39               | 0.1243            | ~11.0s
Feature Extractor (FC)    | 96.08               | 0.1594            | ~5.8s
```

Observations:

- The fixed feature extractor runs almost twice as fast per epoch (5.8s vs 11.0s) because gradient backpropagation is stopped at the linear classification stage.
- When data is limited and resembles natural ImageNet images, fixed feature extraction gives an efficient model with minimal computational power. When maximum accuracy is essential and fine visual differences must be separately identified, fine tuning the entire network results in a more optimal model.